

AI ASSISTANT CODING

Lab Assignment-9.1

2303A54012

A.Sanjay

B47A

Problem 1:

Consider the following Python

```
function: def find_max(numbers):  
    return max(numbers)
```

Task:

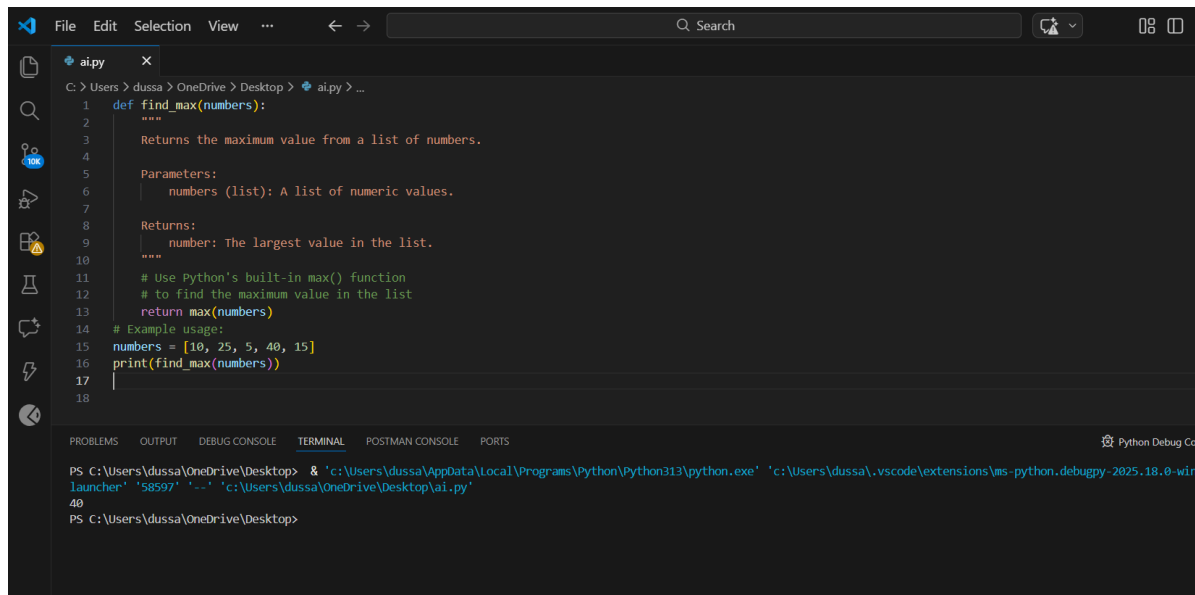
- Write documentation for the function in all three formats:
 - (a) Docstring
 - (b) Inline comments
 - (c) Google-style documentation
- Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.
- Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.

Prompt:

“Generate documentation for the given Python function in three formats: docstring, inline comments, and Google-style documentation. Also compare these documentation styles and recommend the best one for a mathematical utility library.”

give me the code and output

Code:



```
File Edit Selection View ... Search
C:\Users\dussa> OneDrive > Desktop > ai.py > ...
1 def find_max(numbers):
2     """
3     Returns the maximum value from a list of numbers.
4
5     Parameters:
6     numbers (list): A list of numeric values.
7
8     Returns:
9     number: The largest value in the list.
10    """
11    # Use Python's built-in max() function
12    # to find the maximum value in the list
13    return max(numbers)
14
15    # Example usage:
16    numbers = [10, 25, 5, 40, 15]
17    print(find_max(numbers))
18
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POSTMAN CONSOLE PORTS
PS C:\Users\dussa\OneDrive\Desktop> & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win
Launcher' '58597' '-.' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
40
PS C:\Users\dussa\OneDrive\Desktop>
```

Explanation:

The function `find_max()` returns the maximum value from a list of numbers. AI is used to automatically generate different documentation styles to understand how documentation improves code clarity and usability.

- Docstring explains the function purpose, parameters, and return value.
- Inline comments describe the logic within the code.
- Google-style documentation provides a structured and professional format suitable for teams and libraries.

Problem 2:

Consider the following Python function:

```
def login(user, password, credentials):
    return credentials.get(user) == password
```

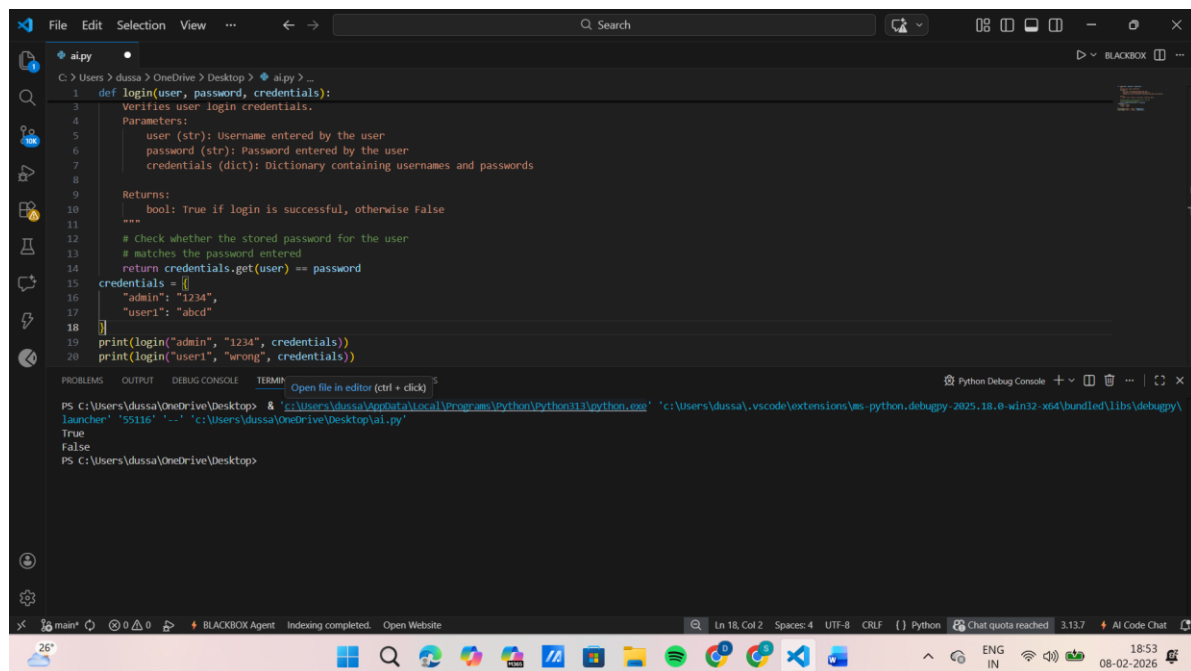
Task:

1. Write documentation in all three formats.
2. Critically compare the approaches.
3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

Prompt:

“Write docstring, inline comments, and Google-style documentation for the given login function. Compare the approaches and recommend the most helpful style for new developers.”

Code:



```
1 def login(user, password, credentials):
2     """
3     Verifies user login credentials.
4     Parameters:
5         user (str): Username entered by the user
6         password (str): Password entered by the user
7         credentials (dict): Dictionary containing usernames and passwords
8
9     Returns:
10         bool: True if login is successful, otherwise False
11     """
12     # Check whether the stored password for the user
13     # matches the password entered
14     return credentials.get(user) == password
15
16 credentials = {
17     "admin": "1234",
18     "user1": "abcd"
19 }
20 print(login("admin", "1234", credentials))
21 print(login("user1", "wrong", credentials))
```

```
PS C:\Users\dussa\OneDrive\Desktop> & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '55116' '-.' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
True
False
PS C:\Users\dussa\OneDrive\Desktop>
```

Explanation:

The login() function checks whether the entered password matches the stored password for a given user.

Different documentation styles help new developers understand authentication logic quickly and safely.

Google-style documentation is especially helpful during onboarding because it clearly defines parameters and return values.

Problem 3:

Calculator (Automatic Documentation Generation)

Task:

Design a Python module named calculator.py and demonstrate automatic documentation generation.

Instructions:

1. Create a Python module calculator.py that includes the following functions, each written with appropriate docstrings:
 - o add(a, b) – returns the sum of two numbers
 - o subtract(a, b) – returns the difference of two numbers
 - o multiply(a, b) – returns the product of two numbers
 - o divide(a, b) – returns the quotient of two numbers
2. Display the module documentation in the terminal using

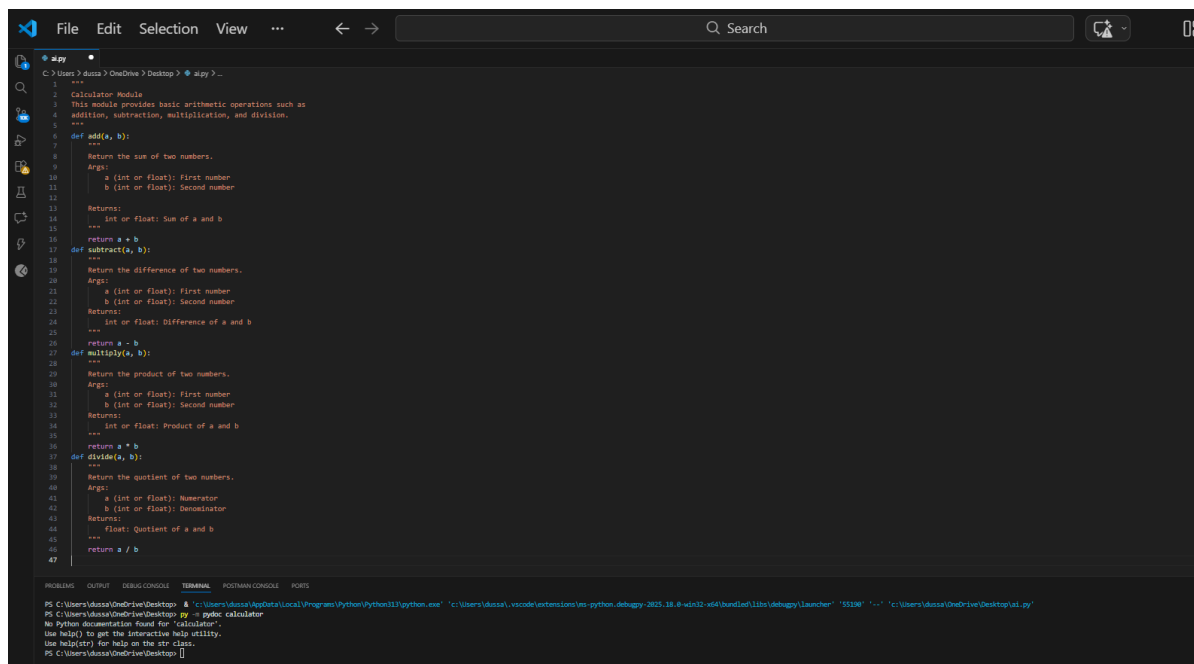
Python's documentation tools.

3. Generate and export the module documentation in HTML format using the pydoc utility, and open the generated HTML file in a web browser to verify the output.

Prompt:

“Create a Python module named calculator.py with arithmetic functions and generate automatic documentation using docstrings and pydoc.”

Code:



```
1  """
2  Calculator Module
3  This module provides basic arithmetic operations such as
4  addition, subtraction, multiplication, and division.
5  """
6  def add(a, b):
7      """
8      Return the sum of two numbers.
9      Args:
10         a (int or float): First number
11         b (int or float): Second number
12     Returns:
13         int or float: Sum of a and b
14     """
15     return a + b
16
17  def subtract(a, b):
18      """
19      Return the difference of two numbers.
20      Args:
21         a (int or float): First number
22         b (int or float): Second number
23     Returns:
24         int or float: Difference of a and b
25     """
26     return a - b
27
28  def multiply(a, b):
29      """
30      Return the product of two numbers.
31      Args:
32         a (int or float): First number
33         b (int or float): Second number
34     Returns:
35         int or float: Product of a and b
36     """
37     return a * b
38
39  def divide(a, b):
40      """
41      Return the quotient of two numbers.
42      Args:
43         a (int or float): Numerator
44         b (int or float): Denominator
45     Returns:
46         float: Quotient of a and b
47     """
48     return a / b
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POSTMAN CONSOLE PORTS

```
PS C:\Users\idusa\OneDrive\Desktop> pydoc calculator
No Python documentation found for 'calculator'.
Use help() to get the interactive help utility.
Use help(str) for help on the str class.
PS C:\Users\idusa\OneDrive\Desktop>
```

Explanation:

The calculator.py module demonstrates how AI-generated docstrings can be used to automatically create documentation.

Python's pydoc tool extracts these docstrings and displays them in terminal and HTML format.

Problem 4:

Conversion Utilities Module

Task:

1. Write a module named conversion.py with functions:

o decimal_to_binary(n)

o binary_to_decimal(b)

- o decimal_to_hexadecimal(n)
- 2. Use Copilot for auto-generating docstrings.
- 3. Generate documentation in the terminal.
- 4. Export the documentation in HTML format and open it in a browser.

Prompt:

“Using Copilot, generate docstrings for number conversion functions and demonstrate automatic documentation generation using pydoc.”

Code:

```

1  C:\Users\dussa> OneDrive\ Desktop > ai.py > ...
2  """
3  Conversion Utilities Module
4  This module provides functions to convert numbers between
5  decimal, binary, and hexadecimal formats.
6  """
7  def decimal_to_binary(n):
8      """Convert decimal number to binary."""
9      return bin(n)[2:]
10 def binary_to_decimal(b):
11     """Convert binary number to decimal."""
12     return int(b, 2)
13
14 def decimal_to_hexadecimal(n):
15     """Convert decimal number to hexadecimal."""
16     return hex(n)[2:]
17

```

```

PS C:\Users\dussa\OneDrive\Desktop> cd 'c:\Users\dussa\OneDrive\Desktop'; & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.de
\libs\debugpy\launcher' '60459' '-c' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
PS C:\Users\dussa\OneDrive\Desktop> py -m pydoc ai
Help on module ai:

NAME
ai - Conversion Utilities Module

DESCRIPTION
This module provides functions to convert numbers between
decimal, binary, and hexadecimal formats.

FUNCTIONS
    binary_to_decimal(b)
        Convert binary number to decimal.

    decimal_to_binary(n)
        Convert decimal number to binary.

-- More --

```

Explanation:

This module converts numbers between decimal, binary, and hexadecimal formats.

AI-generated docstrings ensure consistency and reduce manual documentation effort.

Problem 5 – Course Management Module

Task:

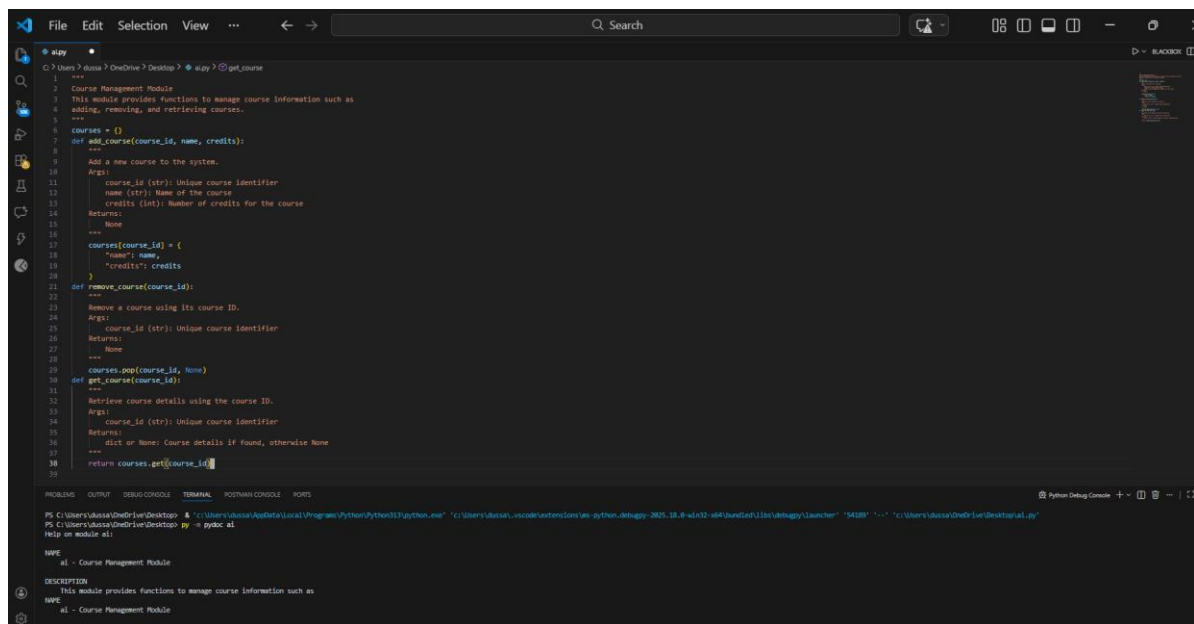
1. Create a module course.py with functions:
 - o add_course(course_id, name, credits)
 - o remove_course(course_id)

- o `get_course(course_id)`
2. Add docstrings with Copilot.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.

Prompt Used:

“Create a course management module with AI-generated docstrings and demonstrate terminal and HTML documentation generation.”

Code:



```
1 """
2 Course Management Module
3 This module provides functions to manage course information such as
4 adding, removing, and retrieving courses.
5 """
6 courses = {}
7
8 def add_course(course_id, name, credits):
9     """
10     Add a new course to the system.
11     """
12     course_id (str): Unique course identifier
13     name (str): Name of the course
14     credits (int): Number of credits for the course
15     Returns:
16     None
17
18     courses[course_id] = {
19         "name": name,
20         "credits": credits
21     }
22
23 def remove_course(course_id):
24     """
25     Remove a course using its course ID.
26     """
27     course_id (str): Unique course identifier
28     Returns:
29     None
30
31     courses.pop(course_id, None)
32
33 def get_course(course_id):
34     """
35     Retrieve course details using the course ID.
36     """
37     course_id (str): Unique course identifier
38     Returns:
39     dict or None: Course details if found, otherwise None
40
41     return courses.get(course_id)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PYTHON CONSOLE PORTS

Python Debug Console

```
PS C:\Users\user\OneDrive\Desktop> cd .\ai.py
PS C:\Users\user\OneDrive\Desktop> pydoc ai
Help on module ai:

NAME
ai - Course Management Module

DESCRIPTION
This module provides functions to manage course information such as

NAME
ai - Course Management Module
```

Explanation:

This module manages course details using basic CRUD operations. AI-generated docstrings improve maintainability and help future developers understand module behavior quickly.